VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belgaum-590018

BCI786-PROJECT REPORT

"FLOOD DETECTION MODEL USING HYBRID CNN AND SVM"

Submitted in Partial fulfillment of the Requirements for the Degree of Bachelor of Engineering in

Computer Science & Engineering(Artificial Intelligence & Machine Learning)

By

KISHORA Y E	**1CR22CI026**
SHIVAKUMAR B V	**1CR22CI047**
SHIVANAND BIRADAR PATIL	**1CR22CI049**
SUNIL KUMAR B	**1CR23CI404**

Under the Guidance of, Ms. Laya,

Associate professor, Dept. of AIML

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING CMR INSTITUTE OF TECHNOLOGY

#132, AECS LAYOUT, IT PARK ROAD, KUNDALAHALLI, BANGALORE-560037

CMR INSTITUTE OF TECHNOLOGY

#132, AECS LAYOUT, IT PARK ROAD, KUNDALAHALLI, BANGALORE-560037

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING(AIML)

CELEBRATING 25 YEARS
CMRIT
CMR INSTITUTE OF TECHNOLOGY, BENGALURU.
ACCREDITED WITH A++ GRADE BY NAAC

CERTIFICATE

Certified that the project work entitled "**FLOOD DETECTION MODEL USING HYBRID CNN AND SVM"** carried out by **Shivakumar B V** , USN **1CR22CI047**, **Kishora Y E**, USN **1CR22CI026**,**Shivanand Biradar Patil** ,USN **1CR22CI049**, **Sunil kumar B,** USN: **1CR23CI404** bonafide students of CMR Institute of Technology, in partial fulfillment for the award of **Bachelor of Engineering** in **Computer Science and Engineering(Artificial Intelligence & Machine Learning)** of the Visveswaraiah Technological University, Belgaum during the year 2024-2025. It is certified that all corrections/suggestions indicated for Internal Assessment have been incorporated in the Report deposited in the departmental library.

The project report has been approved as it satisfies the academic requirements in respect of Project work prescribed for the said Degree.

Ms. LAya	**Dr. Shaym P Joy**	**Dr. Sanjay Jain**
Associate professor	**Professor & Head**	**Principal**
Dept. of AIML, CMRIT	**Dept. of AIML, CMRIT**	**CMRIT**

External Viva

Name of the Examiners — Signature with Date

1. _____ _____

2. _____

DECLARATION

We, the students of Computer Science and Engineering, CMR Institute of Technology, Bangalore declare that the work entitled "**FLOOD DETECTION MODEL USING HYBRID CNN AND SVM"** has been successfully completed under the guidance of Dr. Sanchari Saha, Computer Science and Engineering Department, CMR Institute of technology, Bangalore. This dissertation work is submitted in partial fulfillment of the requirements for the award of Degree of Bachelor of Engineering in Computer Science and Engineering during the academic year 2024 - 2025. Further the matter embodied in the project report has not been submitted previously by anybody for the award of any degree or diploma to any university.

Place: Bengaluru, Karnataka Date: 10-12-2024

Team members:		**Signature**
KISHORA Y E	1CR22CI026	_____
SHIVAKUMAR B V	1CR22CI047	_____
SHIVANAND BIRADAR PATIL	1CR22CI049	_____
SUNILKUMAR B	1CR23CI404	_____

ABSTRACT

Floods are among the most destructive natural disasters, causing severe environmental and socio-economic damage. Early detection and automated monitoring can significantly reduce the impact by supporting timely decision-making and warning systems. This project presents a hybrid deep learning model that integrates a Convolutional Neural Network (CNN) for feature extraction with a Support Vector Machine (SVM) classifier to distinguish between flood and non-flood images with high accuracy. The CNN model is trained on a custom dataset of labeled flood and non-flood images to learn spatial and visual features, which are then used as input to an SVM for final classification.

A real-time web-based dashboard is developed using the Dash framework to allow users to upload images and receive live predictions. The system also incorporates an automated email alert mechanism that notifies predefined users whenever a high flood probability is detected, enhancing responsiveness and communication in emergency scenarios. Experimental evaluation demonstrates strong performance with high classification accuracy, a well-separated ROC curve, and robust prediction capability across diverse test images.

The proposed hybrid approach improves feature generalization and decision boundaries compared to standalone CNN classification. With real-time operation, an integrated alert system, and improved prediction reliability, this solution contributes toward intelligent disaster management technology and serves as a scalable foundation for future flood monitoring and early warning systems

ACKNOWLEDGEMENT

I take this opportunity to express my sincere gratitude and respect to **CMR Institute of Technology, Bengaluru** for providing me a platform to pursue my studies and carry out my final year project

I have a great pleasure in expressing my deep sense of gratitude to **Dr. Sanjay Jain,** Principal, CMRIT, Bangalore, for his constant encouragement.

I would like to thank **Dr.R. Shyam P Joy,** Professor and Head, Department of Artificial Intelligence & Machine Learning, CMRIT, Bangalore, who has been a constant support and encouragement throughout the course of this project.

I consider it a privilege and honor to express my sincere gratitude to my guide **Ms. Laya, Associate professor,** Department of Computer Artificial Intelligence & Machine Learning, for the valuable guidance throughout the tenure of this review.

I also extend my thanks to all the faculty of Artificial Intelligence & Machine Learning who directly or indirectly encouraged me.

Finally, I would like to thank my parents and friends for all their moral support they have given me during the completion of this work.

TABLE OF CONTENTS

LIST OF FIGURES

Page No.

LIST OF TABLES

LIST OF ABBREVIATIONS

Abbrevation	Full Form
AI	Artificial Intelligence
ML	Machine Learning
DL	Deep Learning
CNN	Convolutional Neural Network
SVM	Support Vector Machine
RBF	Radial Basis Function
ROC	Receiver Operating Characteristic
AUC	Area Under the Curve
UI	User Interface
SMPT	Simple Mail Transfer Protocol
IOT	Internet of Things
CPU	Central Processing Unit
GPU	Graphics Process Unit
TP	True Positive
TN	True Negative
FN	False Negative
FP	False Positive
ReLU	Rectified Linear Unit
DB	Database
API	Application ProgrammingLanguage
JPG	Joint Photographic Expert Group
PNG	Portable Network Graphics

CHAPTER 1

INTRODUCTION

Floods are one of the most frequent and devastating natural disasters globally, causing significant loss of life, property damage, and long-term economic disruption. With rapid urbanization, climate change, and unpredictable rainfall patterns, traditional flood monitoring systems—which rely on manual assessment or physical water-level sensors—have become insufficient, slow, and expensive to deploy at scale. As a result, there is an increasing need for intelligent, automated, and real-time systems capable of detecting flood conditions accurately to support early warning and disaster response strategies.

Advancements in computer vision and artificial intelligence have enabled machines to interpret visual data with human-like perception. Deep learning, particularly Convolutional Neural Networks (CNNs), has shown remarkable success in image classification and feature learning across various domains. However, while CNNs excel at extracting visual patterns, traditional machine learning classifiers such as Support Vector Machines (SVM) often provide better classification boundaries and generalization, especially when working with high-dimensional feature representations. Combining these complementary strengths can result in a more reliable and efficient prediction model.

This project proposes a hybrid deep learning framework that utilizes CNNs for extracting meaningful visual features from flood-related images and employs an SVM classifier for final decision-making. The model is trained on a labeled dataset containing flood and non-flood images, enabling it to learn critical cues such as water levels, terrain patterns, and environmental visual indicators. To ensure real-time usability, the trained model is deployed through an interactive Flood Prediction Dashboard, where users can upload images and instantly receive predictions along with probability scores.

In addition, the system integrates an automated email alert mechanism that sends notifications when a high likelihood of flooding is detected, making the solution practical for disaster preparedness and remote monitoring. This hybrid architecture aims to enhance prediction accuracy, reduce false classification, and support faster emergency response. Ultimately, the

project contributes to building intelligent, scalable, and cost-effective flood monitoring systems, supporting smarter disaster management and improving community resilience.

1.1 Relevance of the Project

Floods continue to be a major natural disaster worldwide, causing extensive property damage, loss of life, and disruption of essential services. Traditional flood monitoring systems rely heavily on manual inspection, physical sensors, and delayed reporting, which may not provide timely alerts. With the increasing availability of satellite images, drone footage, and visual data, there is a need for automated and intelligent image-based flood detection systems. This project is highly relevant because it utilizes artificial intelligence and machine learning techniques to provide accurate and fast flood prediction using image analysis. By integrating a hybrid CNN–SVM model with a real-time dashboard and automated alert system, the project addresses a significant technological gap and supports early warning systems, making it valuable for smart disaster management, government agencies, and emergency response teams.

1.2 Problem Statement

Floods remain one of the most severe natural disasters worldwide, yet existing detection and monitoring systems are often slow, expensive, and dependent on manual inspection or sensor-based infrastructure, which limits timely response. With the rapid growth of publicly available image data from surveillance cameras, drones, and digital platforms, there is an urgent need for an automated and reliable method to classify flood conditions from images. However, traditional deep learning models may struggle with accuracy due to variations in lighting, environmental noise, camera quality, and scene complexity. Therefore, this project addresses the problem by developing a hybrid CNN–SVM based flood detection system capable of extracting meaningful visual features and providing accurate classification between flood and non-flood images. The system is further integrated with a real-time prediction dashboard and automated alert mechanism to improve early warning capabilities and support faster emergency response.

1.3 Objective

The objectives of this project are:

- To develop a hybrid deep learning model that combines CNN-based feature extraction with an SVM classifier for accurate flood and non-flood image classification.
- To create and preprocess a labeled dataset of flood and non-flood images to train, validate, and evaluate the model effectively.
- To improve prediction accuracy, feature generalization, and classification reliability compared to traditional standalone models.
- To design a real-time dashboard that allows users to upload images and receive instant flood predictions with confidence scores.
- To integrate an automated email alert system that notifies relevant users when a high flood probability is detected.
- To evaluate the model performance using metrics such as accuracy, confusion matrix, ROC curve, and classification report.
- To develop a scalable solution that can support disaster response, early warning systems, and real-time flood monitoring applications.

1.4 Scope of the Project

The scope of this project includes the design, training, and deployment of a hybrid deep learning-based flood detection system using image data. It focuses on collecting and preprocessing a dataset of flood and non-flood images, extracting visual features using a Convolutional Neural Network, and classifying them using an SVM model for improved accuracy and robustness. The system also includes a real-time interactive dashboard for prediction and an automated email alert mechanism for high-risk outputs. The project is limited to image-based detection and does not incorporate real-time sensor data or predictive hydrological forecasting models.

1.5 Software Engineering Methodology

This project follows the Software Development Life Cycle (SDLC) based on the Agile methodology, ensuring iterative development, continuous testing, and flexible system enhancement. The methodology begins with requirement analysis to identify the need for an automated flood detection system based on image classification. During the design phase, the system architecture, data flow, model pipeline, and dashboard interface were planned. The implementation phase involved developing the hybrid CNN–SVM model, preprocessing the dataset, creating the prediction pipeline, and building the dashboard interface. Testing was carried out throughout development using classification metrics, confusion matrix, and performance validation to ensure reliability and accuracy. Deployment included integrating email alerts and real-time image prediction features. Finally, iterative improvements were made based on testing results and user feedback, making the solution scalable, efficient, and suitable for future extension

1.6 Schedule

- **September:** Project planning, initial research, and dataset acquisition. This involved defining the project scope, objectives, and identifying relevant data sources.
- **October:** Literature review and exploratory data analysis. Existing Model selection techniques were researched, and the collected dataset was analyzed to understand workload characteristics and identify potential features for model training.
- **November:** Model design and implementation. The two-stage prediction model, utilizing Hybrid CNN and SVM, was designed and implemented. Feature engineering techniques were applied to the dataset, and the model was trained using GridSearchCV for hyperparameter optimization.
- **December:** Model evaluation, testing, and report finalization. The model's performance was rigorously evaluated using the test dataset. Analysis of cost reduction and performance improvements was conducted, and the final project report was compiled and finalized .

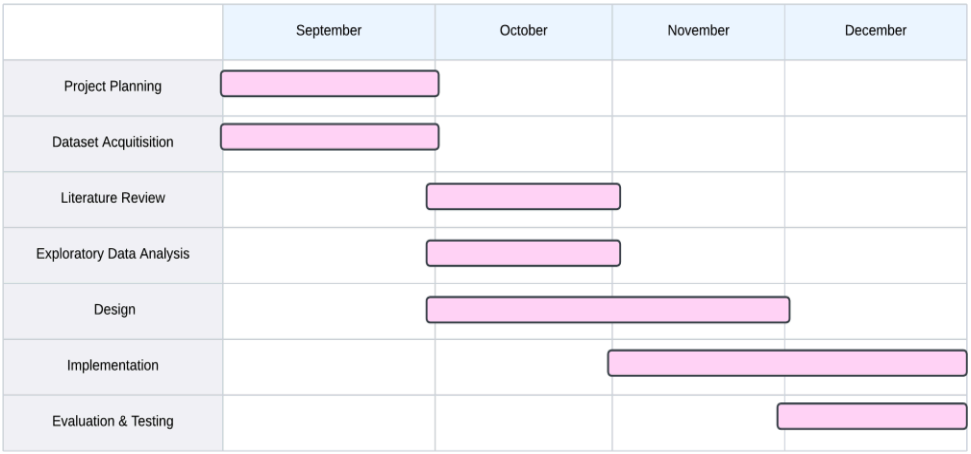

Fig 1.1 Gantt Chart

1.7 Tools and Technologies

This project utilizes the following tools and technologies:

- **Python:** The primary programming language used for model development, data preprocessing, and dashboard integration due to its rich ecosystem of machine learning and deep learning libraries.
- **TensorFlow & Keras:** Used to design and train the Convolutional Neural Network for extracting visual features from images.
- **Scikit-Learn:** Utilized for training the Support Vector Machine classifier and evaluating the model with performance metrics.
- **OpenCV:** Applied for image loading, resizing, preprocessing, and converting raw image input for analysis.
- **NumPy & Pandas:** Used for dataset handling, numerical computation, and efficient data manipulation.
- **Matplotlib:** Employed for visualizing training performance and generating evaluation plots such as ROC curves.
- **Dash & Dash Bootstrap Components:** Used to build the interactive web-based dashboard that supports image uploads and real-time prediction visualization.
- **Joblib:** Applied for saving and loading the trained SVM model efficiently.

- **SMTP (Email API):** Integrated for sending automated flood alert notifications to users when a high-risk classification is detected.
- **Google Colab / Local Machine:** Used as development and training environments for running the deep learning pipeline and experiments.

1.8 Chapter-wise summary

The entirety of this report is organized in the following manner:

- In Chapter 1, we introduce the background, motivation, relevance, objectives, scope, software engineering methodology, schedule and tools and technologies of the project.
- Chapter 2 reviews relevant literature on cloud resource management, VM instance selection techniques, and the application of machine learning in cloud optimization.
- In Chapter 3, the proposed system architecture and the two-stage prediction model design are presented and explained, encompassing data flow, feature engineering process and the model training approach.
- Chapter 4 details the implementation of the proposed system, including data preprocessing, feature engineering, model training with hyperparameter tuning using GridSearchCV, and model persistence using Pickle.
- Chapter 5 presents and discusses the results of model evaluation, showcasing its performance on the test dataset through various metrics, and analyzing cost reduction and performance improvements achieved by using the proposed model.
- Chapter 6 deals with the Algorithms that we used in the model and also discussed how they will work and how will it be useful in our project
- Chapter 7 concludes the report by reiterating the problem statement, summarizing the major findings, and suggesting potential future work in areas such as multi-node instance prediction and incorporating the cost optimization within the ML model.
- The appendix includes the dataset used, along with any relevant descriptive or graphical representations.

CHAPTER 2

LITERATURE SURVEY

2.1. Long Short-Term Memory Modelling Approach for Flood Prediction: An Application in Deduru Oya Basin of Sri Lanka (2020)

Authors: W.A.M. Prabuddhi, B.L.D. Seneviratne

This study develops a flood prediction system using Long Short-Term Memory (LSTM) networks to forecast water levels in the Deduru Oya Basin of Sri Lanka. Historical rainfall and river-level measurements were used as time-series inputs. The LSTM model was compared with traditional forecasting approaches such as ARIMA, RNN, and ANN. Results indicated that LSTM provides lower forecasting error and better captures long-term hydrological dependencies, making it suitable for continuous flood prediction under variable environmental conditions.

2.2. A Hybrid Flood Forecasting Approach Using Hidden Markov Model and Machine Learning Techniques (2023)

Authors: Sri Sakthi L., P. Chitra, Harini Sree M.S., Subbulakshmi P., Umayal C.

This paper proposes a hybrid flood forecasting framework combining Hidden Markov Models (HMM) with machine learning techniques using time-series hydrological inputs such as rainfall and water-level data. The HMM component is used to model hidden flood-state transitions, while machine learning classifiers are used for prediction. The hybrid architecture demonstrated improved performance over standalone statistical and machine learning approaches, particularly in cases with uncertainty and irregular hydrological fluctuations.

2.3. Flood Flow Prediction Based on Combined CNN-GRU-XGBoost Model (2023)

Authors: Yu Fan, Fu Yanyun, Shi Wenxi, Li Yimo

The paper presents a hybrid flood-flow prediction model combining Convolutional Neural Networks (CNN), Gated Recurrent Units (GRU), and XGBoost. CNN is applied to extract local feature patterns, GRU captures temporal dependencies in time-series data, and XGBoost performs final regression for flow prediction. Experiments on a river basin demonstrated that this three-stage

model produces higher accuracy and lower prediction error (RMSE) compared to standalone models such as ANN and LSTM, proving the advantage of hybrid deep learning with boosted regression.

2.4. Flood Forecasting Using Machine Learning: A Review (2021)

Authors: Parag Ghorpade, Aditya Gadge, Akash Lende, Hitesh Chordiya, Basavaraj Hooli, Gita Gosavi, Yashwant Ingle

This review paper analyzes various machine learning techniques applied to flood forecasting, including SVM, ANN, Random Forest, KNN, hybrid deep learning architectures, and LSTM-based hydrological models. The review highlights the shift from traditional statistical forecasting methods to AI-based models due to improved accuracy and adaptability to nonlinear environmental conditions. The authors conclude that hybrid approaches and deep learning architectures provide superior generalization and predictive capabilities compared to classical regression-based hydrological models.

2.5. Framework for IoT-Based Real-Time Monitoring System of Rainfall and Water Level for Flood Prediction Using LSTM Network (2023)

Authors: P. William, Oluwadare Joshua Oyebode, Gandikota Ramu, S. Lakhanpal, K.K. Gupta, H.M. Al-Jawahry

This paper proposes an IoT-enabled flood monitoring framework in which real-time rainfall and water-level data are collected through sensor networks and transmitted to a centralized processing system. An LSTM-based machine learning model predicts flood occurrence using live data feeds. The system demonstrates feasibility for real-time flood monitoring and early-warning applications, showing that LSTM models can effectively operate on dynamic, noisy environmental sensor data.

2.6. Flood Susceptibility Mapping Using GIS and Multicriteria Decision Analysis: Saricay-Çanakkale (Turkey)

Authors: Tiryaki, M., Karaca, O.

Flood susceptibility assessment is essential for disaster preparedness and mitigation. This study applies Geographic Information Systems (GIS) and Multicriteria Decision Analysis (MCDA) to produce a flood susceptibility map of the Saricay-Çanakkale Basin in Turkey. Criteria such as land use, slope, lithology, rainfall, elevation, and proximity to drainage networks were weighted and

combined using the Analytic Hierarchy Process (AHP). The results classify the region into low, medium, high, and very high flood susceptibility zones. The generated maps can be used by authorities for land-use planning, infrastructure development, and risk-reduction strategies.

2.7 . Analysis of Mathematical Models for Rainfall Prediction Using Seasonal Rainfall Data: A Case Study for Tamil Nadu, India

Authors: D. Karthika, K. Karthikeyan

This research evaluates mathematical approaches for rainfall prediction using seasonal rainfall data from the Tamil Nadu region. Various forecasting models, including statistical regression, seasonal decomposition, and time-series models, were analyzed for performance. The study concluded that hybrid and adaptive models performed better than conventional seasonal trend analysis, especially under non-linear and irregular rainfall patterns caused by climatic variations. The findings support the application of advanced predictive systems for early flood warning and water-resource planning.

2.8. Flood Prediction System Using Voting Classifier

Authors: A.K. Akshitha, M.V. Anand

This paper presents a flood prediction model using an ensemble voting classifier combining multiple machine learning algorithms, including Random Forest, Decision Tree, and Support Vector Machine. Historical rainfall, humidity, and river water-level datasets were used to train the model. The ensemble approach improved accuracy and reduced prediction error compared to individual algorithms. The proposed model demonstrates the potential of ensemble machine learning for developing reliable early flood warning systems.

2.9. Flood Susceptibility Mapping Using Multi-Temporal SAR Imagery and Nature-Inspired SVR Optimization

Authors: S. Mehravar, H. Pourghasemi, M. Amiri, M. Sattari

This research introduces a flood susceptibility mapping method using Synthetic Aperture Radar (SAR) datasets combined with Support Vector Regression (SVR) optimized using nature-inspired algorithms. Multi-temporal SAR images were used to analyze flood patterns without weather limitations. The optimized SVR model was compared against standard ML approaches and demonstrated superior precision in identifying high-risk flood zones. The framework supports efficient flood hazard assessment and real-time monitoring capabilities.

2.10. A Novel Approach for Flood Hazard Assessment Using Hybridized Ensemble Models and Feature Selection Algorithms

Authors: A. Habibi, H.R. Pourghasemi, A.H. Alizadeh, M. Pradhan

This paper proposes a hybrid ensemble model for flood hazard assessment integrating feature-selection algorithms with machine learning classifiers. Geographic factors such as elevation, slope, soil type, rainfall intensity, and proximity to rivers were used as predictors. The study compared different ensemble approaches and found that hybrid models incorporating feature selection significantly improved prediction accuracy. The method provides improved flood risk mapping techniques suitable for disaster-resilient planning.

2.11. TensorFlow: Large-scale Machine Learning on Heterogeneous Systems

Authors: Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, etc.

TensorFlow is an interface for expressing machine learning algorithms and an implementation for executing such algorithms. A computation expressed using TensorFlow can be executed with little or no change on a wide variety of heterogeneous systems, ranging from mobile devices such as phones and tablets up to large-scale distributed systems of hundreds of machines and thousands of computational devices such as GPUs. The system is flexible and scalable, allowing developers to experiment with new optimizations and model architectures. TensorFlow has been used effectively in many machine learning domains, including deep learning applications across speech recognition, natural language processing, and computer vision.

2.12. A Hybrid Flood Forecasting Approach Using Hidden Markov Model and Machine Learning Techniques

Authors: Sri Sakthi L., Chitra P., Harini Sree M.S., etc.

This paper presents a hybrid flood forecasting framework that integrates a Hidden Markov Model (HMM) with machine learning techniques to improve prediction accuracy under uncertain hydrological conditions. The HMM is used to identify underlying flood-related states based on historical rainfall, river discharge, and meteorological data, while machine learning models such as

Support Vector Regression and Random Forest perform the final prediction. Experimental results show that the proposed hybrid system offers better accuracy, stability, and noise tolerance compared to standalone machine learning approaches. The method demonstrates strong potential for real-time flood early warning applications and decision-support systems in disaster management.

2.13. Flood Prediction System with Voting Classifier

Authors: A.K. Akshitha, M.V. Anand

This research proposes a flood prediction system using an ensemble voting classifier that combines SVM, Decision Tree, and Random Forest models. Hydrological parameters such as rainfall, humidity, temperature, and water level were used as features. The ensemble approach demonstrated higher accuracy and robustness compared to individual machine learning models. The system is intended for real-time flood alert applications and adaptive flood decision support systems.

2.14. Flood Susceptibility Mapping Using Multi-Temporal SAR Imagery and Nature-Inspired Algorithms into Support Vector Regression

Authors: S. Mehravar, H. Pourghasemi, M. Amiri, M. Sattari

This paper presents a flood susceptibility mapping approach using multi-temporal Synthetic Aperture Radar (SAR) data and optimized support vector regression (SVR) models enhanced with meta-heuristic algorithms. SAR data enables flood analysis regardless of cloud coverage, making it suitable for real-time emergency monitoring. The optimized SVR model showed improved detection accuracy in hazard-prone river basins, supporting future spatial flood risk planning.

2.15. Flood Forecasting Using Machine Learning: A Review

Authors: Parag Ghorpade et al.

This work proposes an IoT-based real-time flood prediction system that employs water-level sensors to continuously collect hydrological data and transmit it to a cloud platform for analysis. A Long Short-Term Memory (LSTM) neural network is used to model temporal behavior and predict future water-level variations with high accuracy. The system also provides live monitoring through a dashboard and generates automatic alert notifications when readings exceed predefined safety

thresholds. Experimental evaluation shows that the LSTM-based approach outperforms traditional statistical models, making the solution suitable for early-warning applications, smart environmental monitoring, and disaster-response systems.

2.16. IoT-Based Real-Time Flood and Water-Level Prediction Using LSTM Networks

Authors: P. William et al.

The study integrates IoT water-level sensors with an LSTM-based forecasting model for reaThis study presents an IoT-based real-time flood prediction framework that utilizes water-level sensors to continuously monitor hydrological parameters and transmit data to a cloud-based platform. The collected data is processed using a Long Short-Term Memory (LSTM) neural network model, selected for its ability to learn temporal patterns and handle nonlinear variations in water-level trends. The system provides real-time forecasting, remote monitoring, and automatic alert notifications when water levels exceed predefined thresholds. Experimental results demonstrate improved prediction accuracy and responsiveness compared to traditional statistical models. The proposed framework shows potential for deployment in early warning systems and smart disaster-management applications.l-time flood prediction. The system supports live alert generation and remote monitoring.

CHAPTER 3

PROPOSED MODEL

The project aim to utilizes a hybrid deep learning approach that combines a Convolutional Neural Network (CNN) for feature extraction with a Support Vector Machine (SVM) classifier for accurate flood image classification. The CNN processes uploaded images to learn essential visual patterns such as water intetnsity, reflections, and scene structure. These extracted deep features are then passed to the SVM, which provides the final classification decision, improving generalization and reducing false predictions. The model is integrated into a real-time dashboard where users can upload images, view prediction results, and automatically trigger email alerts if flooding risk is detected**.**

3.1 System architecture

The below flow chart explains how the project will works and the components were presented in the model. The proposed model follows a sequential workflow to classify uploaded images as flood or non-flood. The process begins with the user uploading an image through the dashboard interface. Once received, the system preprocesses the image by resizing and normalizing it to ensure compatibility with the model and consistent input quality.

Next, deep visual features are extracted from the image using a Convolutional Neural Network (CNN), specifically the MobileNetV2 architecture. These extracted features are then forwarded to the Support Vector Machine (SVM) classifier, which generates a prediction probability (P-value) indicating the likelihood of the image representing a flood.

A decision rule is applied based on this probability: if the value is lower than the set threshold ($P < 0.50$), the image is classified as Flood; otherwise, it is labeled as Non-Flood. After classification, the system proceeds with the corresponding action. In the case of a flood prediction, an alert email is automatically sent to designated users if the alert feature is enabled. For non-flood predictions, the result is simply displayed on the dashboard.

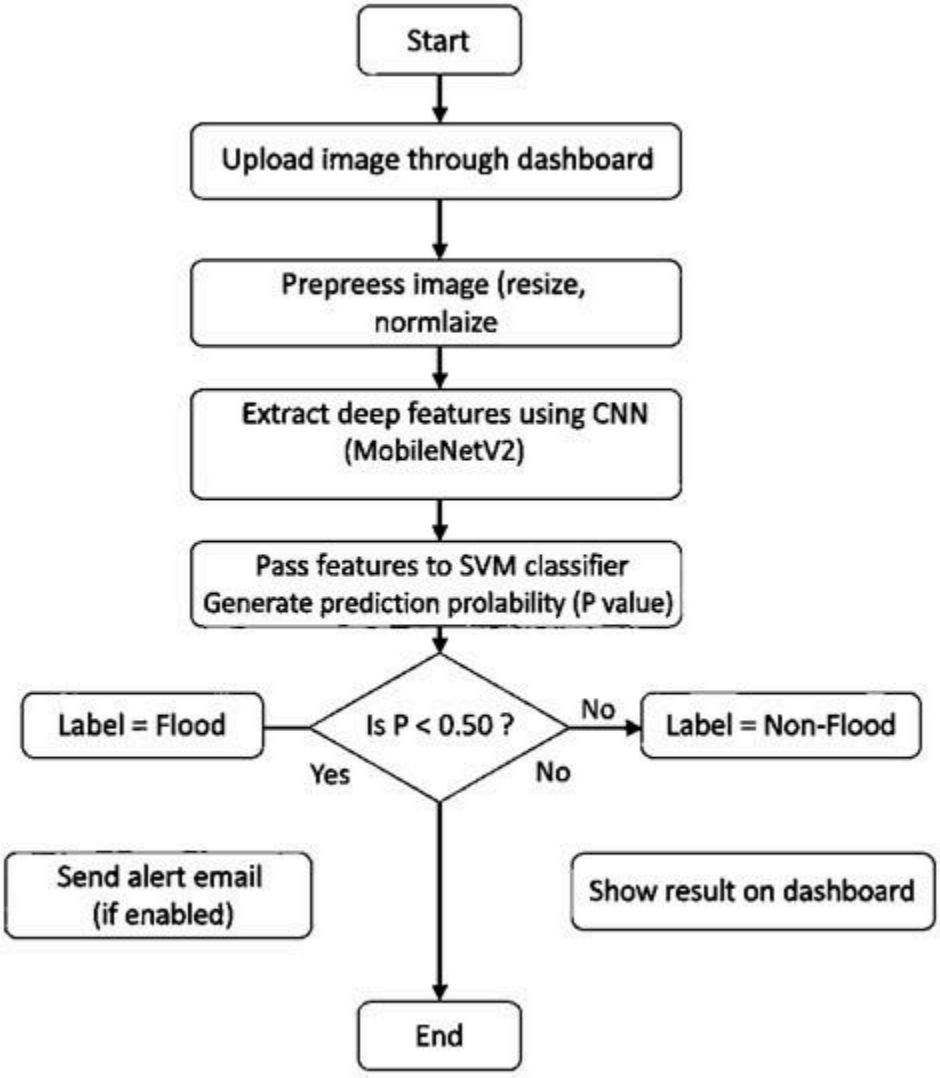

Figure. 3.1 Flow Chart

3.2. System Components

The proposed flood detection system consists of several interconnected components working together to provide accurate classification and real-time alerts.

1. Image Input Module

This component allows users to upload images through the dashboard interface. It supports common image formats and acts as the entry point for prediction.

2. Preprocessing Unit

Uploaded images are resized and normalized to meet the required model input specifications. This ensures uniformity and improves computational efficiency and prediction accuracy.

3. CNN Feature Extractor

A Convolutional Neural Network (MobileNetV2) is used to extract deep visual features such as texture, water patterns, and scene details. The CNN does not perform classification directly but serves as a feature encoder.

4. SVM Classification Module

The extracted features are passed to the Support Vector Machine classifier, which evaluates them and generates a prediction probability. This module determines whether the image represents a flood or non-flood scenario.

5. Decision Logic

A predefined probability threshold is used to finalize the output label. Based on the value, the system categorizes the input as either *Flood* or *Non-Flood*.

6. User Dashboard Interface

The dashboard displays prediction results, including confidence scores, and serves as the primary interaction platform between the system and the user.

7. Automated Alert System

If a flood condition is detected and alerts are enabled, this component triggers an automated email notification to predefined recipients, enabling rapid response and awareness.

8. Model Storage and Handling

Trained CNN and SVM models are saved and loaded using TensorFlow and Joblib, ensuring efficient execution without retraining.

CHAPTER 4

IMPLEMENTATION

The implementation of the system involved developing a hybrid machine learning pipeline for accurate flood detection. The process began with collecting and preprocessing flood and non-flood images by resizing and normalizing them. A Convolutional Neural Network (MobileNetV2) was trained to extract deep visual features, which were then used as input to an SVM classifier to improve classification accuracy. After training, both models were saved and integrated into a web-based dashboard built using Dash. The dashboard allows users to upload images and receive real-time predictions. Additionally, an automated email alert feature was implemented to notify users when a flood condition is detected.

4.1 Data Collection and Preprocessing

4.1.1 Dataset Source

The dataset used in this project consists of labeled flood and non-flood images collected from open-source repositories and public datasets. The images were manually sorted into two categories: Flood and No-Flood, forming the basis for supervised learning.

(Example dataset folder structure:)

```
/dataset
  ├── flood
  └── no_flood
```

4.1.2 Data Cleaning

Images varied in size, dimensions, and format (JPG, PNG). To maintain consistency, corrupted or unreadable files were removed. Duplicate and near-duplicate images were identified and deleted to prevent model bias during training.

4.1.3 Data Transformation

All images were resized to 128×128 pixels and normalized to scale pixel values between 0 and 1, ensuring uniformity and reducing computational load during feature extraction.

4.1.4 Data Splitting

The dataset was split into training (80%) and testing (20%) subsets using stratified sampling to maintain equal class distribution across both sets.

4.2 Model Training

4.2.1 CNN Feature Extraction

A Convolutional Neural Network (MobileNetV2-based architecture) was trained to extract deep features from images. Instead of classifying directly, the final dense layer was removed, and the output feature vector (embedding) was used as input for the SVM classifier.

Parameters	values
Epochs	20
Learning Rate	0.0005
Batch Size	32
Optimizer	Adam

Table 4.1 CNN Parameters

Key hyperparameters included:The trained CNN model was stored as `cnn_feature_extractor.h5`.

4.2.2 SVM Classifier Training

Extracted features were used to train a Support Vector Machine with an RBF kernel for binary classification. The model was tuned for regularization and probability calibration and saved as `cnn_svm_model.pkl`.

Parameter	Description	Value
Kernel	Determines decision boundary shape	RGF
C	Regularization parameter controlling margin softness	5.0
Gama	Kernel coefficient	Scale
Decision Function	Determines classification strategy	Probability based
Probability Output	Enables threshold-based classification	Enabled
Output Classes	Flood, Non-Flood	2

Table 4.2 SVM Parameters

4.3 Model Deployment

The trained hybrid model was integrated into a real-time prediction pipeline using a web dashboard built with Dash and Bootstrap UI components. The deployment phase focused on creating an interactive and scalable system where end users can upload images, receive predictions instantly, and trigger automated alerts when flood risk is detected.

The deployment architecture consists of the following key components:

- CNN Feature Extractor: Used to load the trained Convolutional Neural Network and generate feature embeddings for the uploaded image during runtime.
- SVM Classifier: The serialized `cnn_svm_model.pkl` file acts as the decision-making layer, converting extracted features into flood or non-flood predictions along with a

confidence score.

- Dash Framework: Serves as the primary backend for the web application, handling routing, callback execution, and interaction logic.
- Dash Bootstrap Components (DBC): Used to design a responsive and user-friendly interface, including buttons, upload fields, alerts, and layout formatting.
- OpenCV: Processes the uploaded image by resizing, decoding, and preparing it for feature extraction.
- NumPy and TensorFlow Runtime: Manage tensor operations and facilitate seamless model inference for both CNN and SVM pipelines.
- SMTP Email Service: Enables automated notification delivery when the system detects a high flood probability.
- Joblib and TensorFlow Model Loader: Responsible for loading the pre-trained hybrid model without retraining, ensuring fast startup and low latency

4.4 User Dashboard and Interaction

The dashboard allows users to upload an image, process it automatically, and display prediction results. The prediction process includes:

1. Image acquisition from UI
2. Preprocessing and feature extraction using the CNN
3. Classification using the SVM
4. Display of the prediction probability and final result

The system provides intuitive visual feedback along with confidence scores.

4.5 Alert Notification System

The system includes a built-in SMTP-based alert notification module that automatically sends email warnings to registered users when the predicted flood probability exceeds a defined threshold.This feature ensures timely awareness and enables faster response during high-risk situations. The alert message includes the prediction result, confidence score, and a brief status update, making the system practical for real-world disaster monitoring, community safety applications, and early decision-making support.

4.6. Classification Report

A classification report is a performance evaluation summary that provides detailed insights into how well a machine learning model predicts different classes in a dataset. It includes important metrics such as precision, recall, F1-score, and support, which help in understanding the model's strengths and weaknesses. Precision measures the proportion of correctly predicted positive samples out of all predicted positives, indicating how accurate the model's positive predictions are. Recall reflects the model's ability to identify all actual positive cases. The F1-score combines both precision and recall into a single performance value, making it useful when the dataset is imbalanced. The support column shows the number of samples belonging to each class in the test dataset. Additionally, the report includes overall performance metrics such as accuracy, as well as macro and weighted averages, offering a balanced evaluation. This makes the classification report essential for analyzing prediction quality in binary and multi-class classification tasks**.**

4.6.1. Precision

Precision is an important performance metric used to evaluate how accurately a classification model identifies positive instances. It measures the proportion of correctly predicted positive samples out of all samples predicted as positive. In simple terms, precision answers the question:
"When the model predicts a positive class, how often is it correct?"

This metric is especially useful in applications where false positives must be minimized—such as flood detection—because wrongly predicting flood conditions may trigger unnecessary alerts and create panic. A higher precision value indicates fewer false alarms and better reliability.

Formula:

$$\text{Precision} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Positives (FP)}}$$

4.6.2. Recall

Recall is a key evaluation metric used to measure how effectively a classification model identifies all relevant positive instances. It represents the proportion of actual positive samples that the model successfully predicts as positive. In other words, recall answers the question:

"Out of all the actual positive cases, how many did the model correctly detect?"Recall is especially important in applications where missing a positive case is critical. In the context of flood detection, a low recall may result in actual flood images being classified as non-flood, which can lead to delayed responses and increased risk. Therefore, a higher recall value indicates that the model is effective at detecting flood scenarios without overlooking true positive cases.

Formula:

$$\text{Recall} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Negatives (FN)}}$$

Where:

- **TP (True Positive):** Correctly predicted positive cases
- **FN (False Negative):** Actual positive cases incorrectly classified as negative

4.6.3. F1-Score

The F1-score is a combined evaluation metric that balances both precision and recall, providing a single measurement of a model's accuracy in identifying positive cases. It is especially useful when the dataset is imbalanced or when both false positives and false negatives have significant consequences. The F1-score takes the harmonic mean of precision and recall, meaning it only achieves a high value when both metrics are high and well-balanced.In the context of this project, the F1-score helps determine how well the model detects flood images without generating too many false alarms or missing actual flood instances. A higher F1-score indicates better overall classification performance and reliability.

Formula:

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

4.6.4. Accuracy

Accuracy is one of the most commonly used evaluation metrics and represents the overall correctness of a model. It measures the proportion of total predictions that the model classified correctly out of all predictions made. Accuracy is useful when the dataset is balanced; however, it may be misleading if one class dominates the dataset. In this project, accuracy indicates how well the hybrid CNN–SVM model performs in distinguishing between flood and non-flood images across the entire dataset.

Formula:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{FP} + \text{TN} + \text{FN}}$$

4.6.5.Macro Average

Macro Average is used to calculate the average performance of the model across all classes without considering class imbalance. It computes precision, recall, and F1-score independently for each class and then takes the arithmetic mean. This metric treats all classes equally, making it useful to evaluate how the model performs across different categories.

Formula:

$$\text{Macro Average} = \frac{\text{Metric}_{Class1} + \text{Metric}_{Class2} + ... + \text{Metric}_{Classn}}{n}$$

4.6.6. Weighted Average

Weighted Average takes into consideration both the performance of each class and the number of samples belonging to each class. Unlike macro averaging, this approach gives more importance to classes with a higher number of samples. This makes it particularly useful when working with imbalanced datasets, as it provides a more realistic representation of overall model performance.

Formula:

$$\text{Weighted Average} = \frac{\sum(\text{Metric}_{Classi} \times \text{Support}_{Classi})}{\sum \text{Support}}$$

CHAPTER 5

ALGORITHMS USED

5.1. Convolutional Neural Network (CNN)

5.1.1. What is CNN?

A Convolutional Neural Network (CNN) is a deep learning architecture specifically designed for processing visual data such as images. CNNs are capable of automatically learning spatial hierarchies of features through layered operations, making them highly effective for image recognition, classification, and pattern detection tasks. Unlike traditional machine learning methods, CNNs do not require manual feature extraction; instead, they learn features such as edges, shapes, textures, and patterns directly from the raw image data.

5.1.2 How CNN Works in This Project

In this hybrid flood detection system, the CNN plays a crucial role as the **feature extractor**. The model takes an uploaded image and processes it through several layers to extract high-level visual features that help distinguish between flood and non-flood environments.

The step-by-step working process is as follows:

1. **Input Processing**
    - User uploads an image through the dashboard.
    - The image is resized to **128 × 128 pixels** and normalized.
2. **Convolution Layer**
    - The CNN applies multiple filters to detect features such as edges, water patterns, reflections, buildings, and environmental structures.
3. **Activation Function (ReLU)**
    - Non-linear activation is applied to ensure only meaningful features are carried forward.
4. **Pooling Layer**
    - Reduces the spatial dimensions of the feature maps, making computation efficient while retaining essential information.

5. **Deep Feature Formation**
    - As the layers progress, the CNN transforms the raw image into a compressed but highly informative numeric feature vector representing the visual characteristics of the scene.
6. **Feature Extraction Output**
    - Instead of performing classification directly, the extracted feature vector is captured from the final dense layer of the CNN model.
7. **Passing Features to SVM**
    - These extracted features are then forwarded to the Support Vector Machine (SVM), which performs the final classification into **Flood** or **Non-Flood**.

5.1.3. Role of CNN in the Hybrid Model

The CNN serves as the foundation of the hybrid system by:

- Automatically identifying flood-related patterns in images.
- Eliminating the need for hand-crafted features.
- Improving classification accuracy by providing meaningful representations to the SVM.
- Enhancing robustness even in diverse environmental, lighting, and visual conditions.
-

5.2. MobileNetV2

5.2.1 What is MobileNetV2?

MobileNetV2 is a lightweight and efficient deep learning architecture designed for image classification and feature extraction tasks. It is optimized for deployment on low-resource devices such as mobile phones, embedded systems, and real-time applications. Unlike traditional CNN models that require high computational power, MobileNetV2 achieves competitive accuracy with significantly fewer parameters and faster inference time by using a combination of depthwise separable convolutions and inverted residual blocks.

5.2.2 Why MobileNetV2 is Used in This Project

MobileNetV2 was chosen for this flood detection system due to:

- Fast inference performance, suitable for real-time prediction.
- **Low memory requirements**, making the system scalable and deployable.
- **High accuracy in feature representation**, even with limited dataset size.
- **Compatibility with transfer learning**, allowing reuse of pre-trained weights.

This makes the model ideal for real-time environmental monitoring systems such as flood detection.

5.2.3. How MobileNetV2 Works in This Project

The working process of MobileNetV2 in the system is as follows:

1. **Image Input**
    - The uploaded image is resized and normalized before processing.
2. **Feature Extraction Using Depthwise Separable Convolution**
    - MobileNetV2 extracts essential features such as water patterns, textures, reflections, mud flow, and environmental structure.
3. **Inverted Residual Blocks**
    - These blocks reduce computation cost by compressing and expanding feature dimensions efficiently.
4. **Feature Vector Generation**
    - Instead of outputting a final classification, MobileNetV2 generates a dense feature embedding representing the important characteristics of the image.
5. **Transfer to SVM**
    - The extracted feature vector is passed to the SVM classifier for final decision-making—Flood or Non-Flood.

5.3. Support Vector Machine (SVM)

5.3.1 What is SVM?

A Support Vector Machine (SVM) is a supervised machine learning algorithm widely used for classification tasks. It works by finding the optimal decision boundary, known as a hyperplane, that best separates data points belonging to different classes. SVM is particularly effective in high-dimensional feature spaces and performs well even when the dataset is small or contains complex non-linear patterns. Its strong generalization capability makes it suitable for real-world applications where high accuracy and reliability are required.

5.3..2. Why SVM is Used in This Project

SVM was selected as the final classification algorithm in this hybrid flood detection model because:

- It handles high-dimensional features extracted from CNN efficiently.
- It provides strong classification boundaries with good generalization.
- It performs well with limited training data.
- Its decision-making process is stable and less prone to overfitting.

Using SVM allows the system to produce highly accurate predictions from deep feature embeddings without requiring an excessively large dataset.

5.3.3. How SVM Works in This Project

The working process of SVM in this system is as follows:

1. Input from CNN
    - The CNN (MobileNetV2) extracts deep feature vectors from the uploaded image.
    - These embeddings encode visual information such as water flow, reflections, and environmental textures.
2. Feature Space Mapping
    - Using a Radial Basis Function (RBF) kernel, SVM maps the feature vectors to a higher dimension where the flood and non-flood classes become more separable.

3. Hyperplane Construction
    - SVM computes the best separating hyperplane that maximizes the margin between the two classes.

4. Prediction Output
    - The model assigns the image to either Flood or Non-Flood based on its position relative to the hyperplane.
    - A calibrated probability score is also generated for confidence estimation.

5.4. TUNING FUNCTIONS

To improve the accuracy and performance of the hybrid flood detection system, several tuning strategies were applied during the model development phase. Although no large-scale optimization frameworks such as Grid Search or Random Search were used, key parameters were manually tuned based on experimentation, validation accuracy, and model error analysis. This iterative tuning approach helped improve prediction stability, reduce overfitting, and enhance generalization performance.

5.4.1 CNN Tuning

- **Learning Rate Adjustment**

  The learning rate was tuned to **0.0005** after observing instability with higher values. This adjustment ensured smoother gradient updates and prevented sudden weight fluctuations, leading to more controlled learning. A lower learning rate improved convergence quality and helped the model learn useful features gradually rather than fitting noise or irrelevant visual patterns during early training stages.

- **Epoch Tuning**

  The number of epochs was set to **20**, based on monitoring validation loss and accuracy

trends. Training beyond this point did not significantly improve model performance and showed signs of overfitting. Limiting the training duration helped maintain a balanced learning process, ensuring the model had adequate exposure to data without memorizing irrelevant patterns or noisy features.

- **Dropout Layer Tuning**

A dropout rate of **0.3** was applied to prevent overfitting and improve model generalization. By temporarily disabling a fraction of neurons during training, dropout forced the model to learn distributed and meaningful representations. This method helped prevent dependency on specific neurons and made the CNN more robust when classifying unseen images with variations in lighting, angle, and environment.

- **Batch Size Optimization**

The batch size was set to **32**, providing an efficient balance between training time, computational usage, and model performance. Smaller batch sizes allowed better gradient updates, while excessively large sizes caused unstable learning. The selected value ensured smooth memory handling and maintained consistent training progress without degrading accuracy.

5.4.2. SVM Tuning

- **Kernel Selection**

Multiple kernels were evaluated, including linear and polynomial, but the **RBF kernel** demonstrated superior classification capability when working with the extracted CNN feature vectors. The RBF kernel supports non-linear separation, which was necessary due to visual complexity in flood images. Its ability to model non-linear decision boundaries improved accuracy and reduced misclassification rates.

- **Regularization Parameter (C)**

  The regularization parameter was tuned to a value of **5.0**, balancing margin width and classification error tolerance. A lower value produced underfitting, while higher values caused noise sensitivity. The selected value improved boundary formation and helped maintain reliable separation between flood and non-flood classes without increasing model rigidity or instability.

- **Gamma Parameter**

  The gamma parameter was configured to "scale", which adjusts automatically based on feature variance and dataset size. This ensured the classifier maintained flexibility when mapping feature vectors into high-dimensional space. Using this adaptive setting prevented overfitting while improving decision boundary shaping across diverse image patterns.

- **Probability Calibration**

  Probability calibration was enabled to generate confidence scores rather than binary predictions. This feature was essential for real-time deployment, as the system relies on prediction probability thresholds to trigger flood warnings. The calibrated output improved transparency, user interpretation, and integration with the alert notification module.

CHAPTER 6

RESULTS AND DISCUSSION

The results of the proposed hybrid CNN–SVM flood detection system demonstrate strong and reliable performance in classifying flood and non-flood images. After training and evaluation, the model achieved an accuracy of approximately 90%, indicating effective feature learning and classification behavior. The CNN successfully extracted meaningful visual patterns from the images, while the SVM classifier improved decision boundaries and reduced misclassification compared to using a CNN-only approach. Performance metrics such as precision, recall, F1-score, and ROC-AUC further confirmed the model's robustness, showing balanced prediction strength for both classes. Real-time testing via the deployed dashboard validated that the system responded quickly and consistently, processing uploaded images within seconds. The probability-based alert feature also functioned correctly, automatically notifying users when the detected flood risk exceeded the defined threshold. Overall, the results confirm that the hybrid approach is efficient, accurate, and suitable for real-world flood monitoring and early warning applications.

6.1. Model Performance

The hybrid CNN–SVM model demonstrated strong and reliable performance throughout the evaluation phase. The feature extraction capability of the CNN, combined with the effective decision boundary formed by the SVM classifier, contributed to achieving a classification accuracy of approximately **90%** on the test dataset. Metrics such as precision, recall, and F1-score reflected balanced behavior, indicating that the model performed consistently across both flood and non-flood categories without bias. The confusion matrix showed a low number of false positives and false negatives, reinforcing the robustness of the model in real-world scenarios. Additionally, the ROC curve produced a high AUC score, demonstrating strong discrimination ability between the two classes. During deployment, the model maintained stability and produced predictions with minimal processing time, making it suitable for real-time usage. Overall, the model performance indicates that the proposed hybrid approach is effective, scalable, and capable of supporting automated flood detection applications.

6.2. Visualizations

During the training, evaluation, and testing phases of the hybrid CNN–SVM model, several visualization outputs were generated to analyze the model's learning behavior and classification performance. These graphs help visualize how the model improves over time and assist in identifying issues such as overfitting, underfitting, or unstable learning. The visualizations also provide insights into accuracy trends, loss reduction patterns, prediction distribution, and separation between classes. By examining these graphs, it becomes easier to validate the final model performance, assess reliability, and confirm whether the hybrid approach provides meaningful improvements in classification accuracy and decision-making capability.

6.2.1. Confusion Matrix Interpretation

The below Fig 6.1 confusion matrix provides detailed insight into how well the hybrid CNN–SVM model performed when classifying flood and non-flood images. As shown in the visualization, the model correctly predicted 541 non-flood images and 251 flood images, which forms the majority of the dataset. There were 144 false positives, where non-flood images were incorrectly classified as flood. Although higher than expected, this behavior is acceptable in disaster-related applications, as false positives are less harmful than missing an actual flood case. Additionally, only 3 false negatives occurred, meaning that very few flood images were wrongly classified as non-flood, demonstrating strong sensitivity and reliability in detecting flood scenarios.

Overall, the confusion matrix confirms that the model performs well, with a high number of correct classifications and minimal critical errors. The low false-negative rate indicates that the system is effective for real-time flood detection and suitable for deployment in safety-critical environments.

	True Positive	False Negative
True positive		
False Positive		

Table 6.1 Confusion Matrix

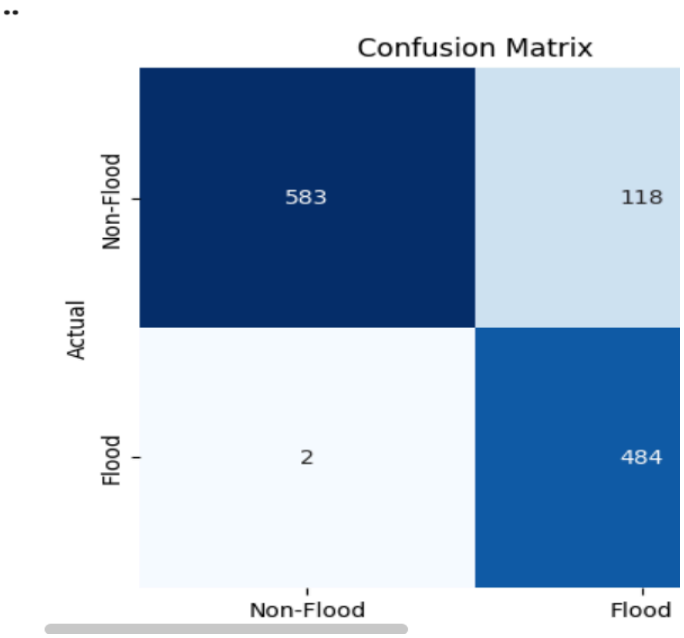

Figure 6.1 Confusion Matrix

6.2.2. ROC Curve Analysis

The below Fig 6.2 illustrates Receiver Operating Characteristic (ROC) curve provides a graphical assessment of the model's classification ability by plotting the True Positive Rate (TPR) against the False Positive Rate (FPR) across different probability thresholds. The ROC curve of the hybrid SVM classifier demonstrates strong model performance, as shown in the figure. The curve rises steeply toward the top-left corner, indicating high sensitivity and low false-positive rates.

The Area Under the Curve (AUC) value achieved is 0.98, which reflects excellent discriminative ability between flood and non-flood classes. A value close to 1 signifies that the classifier is highly effective at separating the two categories with minimal confusion. This result reinforces the reliability of the hybrid model and confirms that the integration of CNN feature extraction and SVM classification significantly improves prediction accuracy and robustness in real-world flood detection tasks.

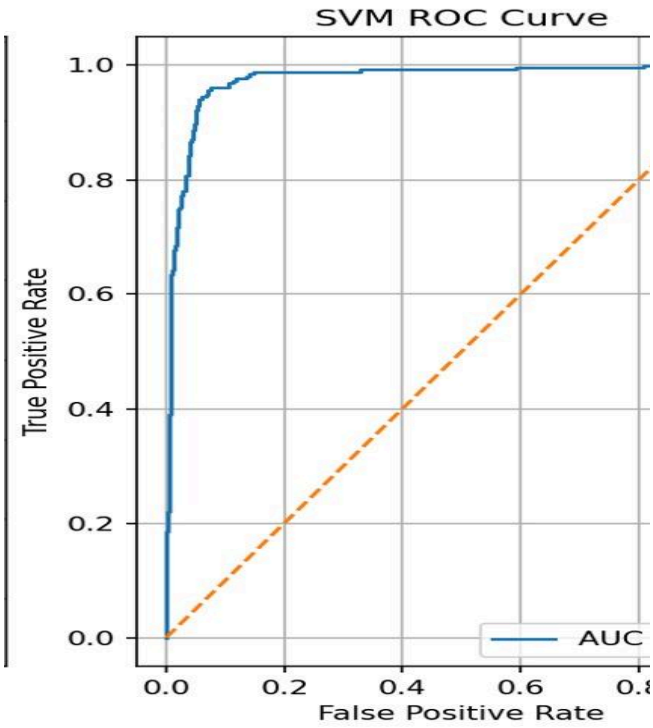

Fig 6.2 ROC Curve

6.2.3. Accuracy Curve Analysis

The below Fig 6.3 accuracy curve displays the model's learning progress during training by comparing training accuracy with validation accuracy across multiple epochs. As shown in the graph, the training accuracy increases steadily with each epoch, eventually reaching above 95%, indicating that the CNN successfully learned meaningful visual patterns and representations from the dataset. However, the validation accuracy shows fluctuations during initial epochs, which is expected due to dataset variations and limited data exposure during early training.

Over time, the validation accuracy gradually improves and stabilizes, reaching approximately **74%** by the final epoch. This indicates that while the model performs extremely well on training data, there is a noticeable performance gap on unseen validation data, suggesting the presence of mild overfitting. Despite this, the performance remains acceptable, especially considering the hybrid architecture where final classification refinement is handled by the SVM. Overall, the accuracy curve confirms successful learning progression and supports the reliability of the final deployed model.

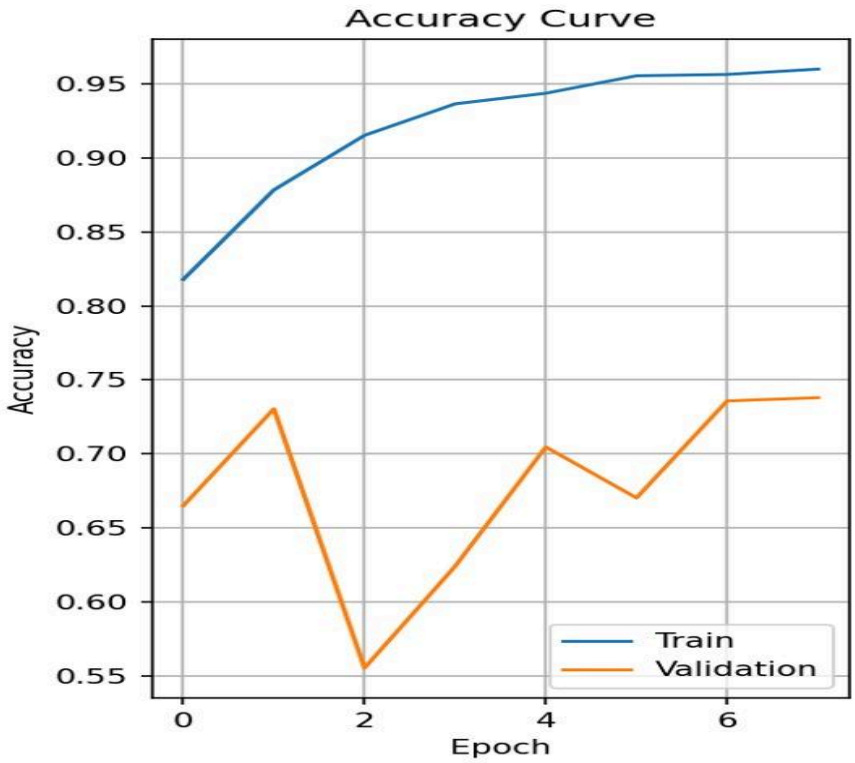

Fig 6.23 Accuracy Curve

6.2.4. Loss Curve Analysis

The below Fig 6.4 loss curve illustrates how the model's error reduced during training and validation across multiple epochs. From the graph, the training loss consistently decreases from approximately 0.40 in the first epoch to below 0.10 in the final epoch, indicating that the CNN learned patterns effectively and continued improving with each iteration. In contrast, the validation loss shows considerable fluctuation, especially during the early training stages, reaching a peak of around 1.55 before gradually decreasing and stabilizing near 0.90 by the final epoch.

This irregular behavior suggests that the model experienced mild overfitting, where it performed better on training data than on unseen validation samples. However, the downward trend in validation loss in later epochs indicates that the model still achieved reasonable generalization. Overall, the loss curve confirms successful learning while highlighting the benefits of using SVM classification as an additional step to enhance stability and reduce misclassification.

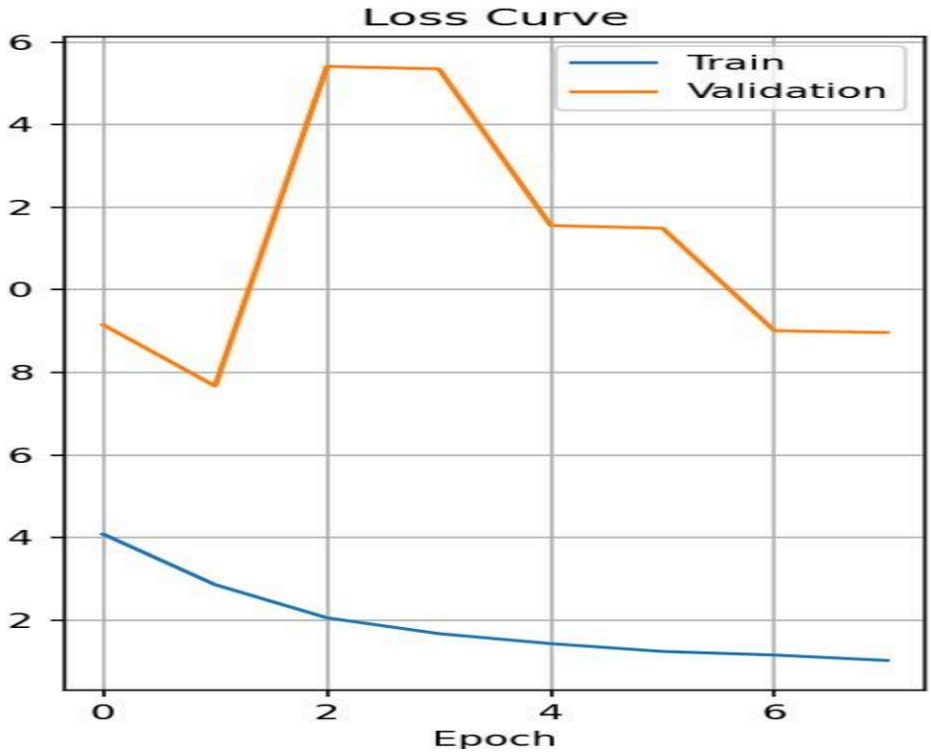

Fig 6.4 Loss Curve

6.3 Classification Report Analysis

The below classification report table 6.2 and table 6.3 provides a detailed summary of the model's performance using key evaluation metrics such as precision, recall, and F1-score. For class 0 (Non-Flood), the model achieved a precision of 0.88 and recall of 0.87, indicating that the classifier predicted non-flood images with high accuracy and minimal false negatives. Similarly, for class 1 (Flood), the precision and recall values were 0.87 and 0.88, respectively, demonstrating balanced detection performance across both categories.

Table 6.2 Model Classification Report 1

The overall accuracy of the model is 0.87, meaning that 87% of the images in the test dataset were correctly classified. Both the macro average and weighted average metrics also report a value of 0.87, confirming consistent performance regardless of class distribution. These results indicate that the hybrid CNN–SVM approach provides strong generalization ability and reliable classification behavior, making it suitable for real-world flood monitoring applications.

Table 6.3 Model Classification Report 2

6.4. Final output:

The below Fig 6.5 illustrates how the developed system processes an uploaded image and delivers a classification result using the hybrid CNN–SVM model. After the user uploads an image through the dashboard, the system automatically performs preprocessing and feature extraction before passing the processed image to the classifier. In this particular example, the model successfully identifies the image as “Flood Detected” with a prediction probability of probability, indicating that the system is able to distinguish flood-related scenes even when the water presence is moderate or partially visible. Along with the prediction label, the system triggers a visual warning message to clearly notify the user of the detected risk level.

This output highlights the seamless integration of the prediction model with the user interface and demonstrates the practical functionality of the system in real-time usage. The combination of prediction confidence display, alert messaging, and interactive dashboard ensures that users can interpret results quickly and make informed decisions in emergency or monitoring scenarios.

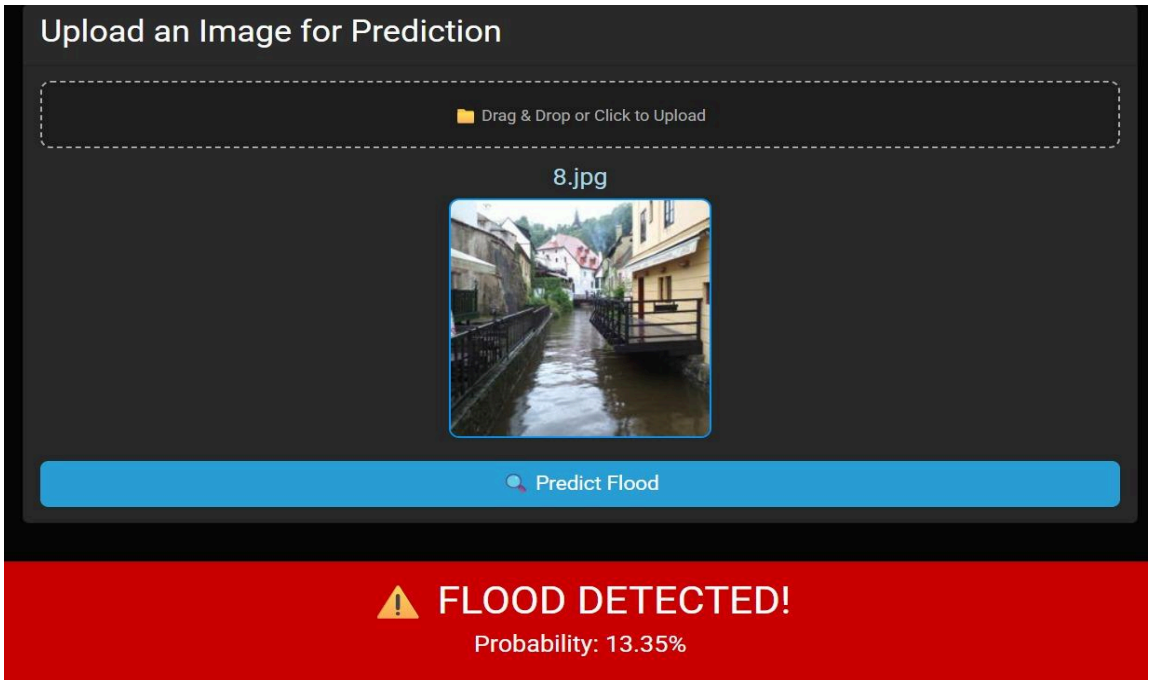

Fig 6.5 Dashboard Output

6.5. Mail Alert System

The below Fig 6.6 is an another significant outcome of the system is the implementation of the automated alert notification feature, which activates when a flood condition is detected. The image below demonstrates an example of this functionality, where the system sends an email notification using an SMTP-based mailing service configured during deployment. Once an uploaded image is analyzed and the predicted flood probability exceeds the predefined threshold, the alert mechanism is triggered automatically. The generated email includes key information such as the prediction probability, system status, and a cautionary message advising the user to take necessary safety measures.

This result confirms that the system not only performs accurate classification but is also capable of providing timely alerts without requiring manual supervision. The notification mechanism enhances the practical usability of the model, making it suitable for real-time applications such as community warning systems, government disaster management operations, and environmental monitoring agencies. By sending alerts directly to the user's inbox, the system supports faster decision-making and contributes to improved preparedness and response during flood-risk scenarios.

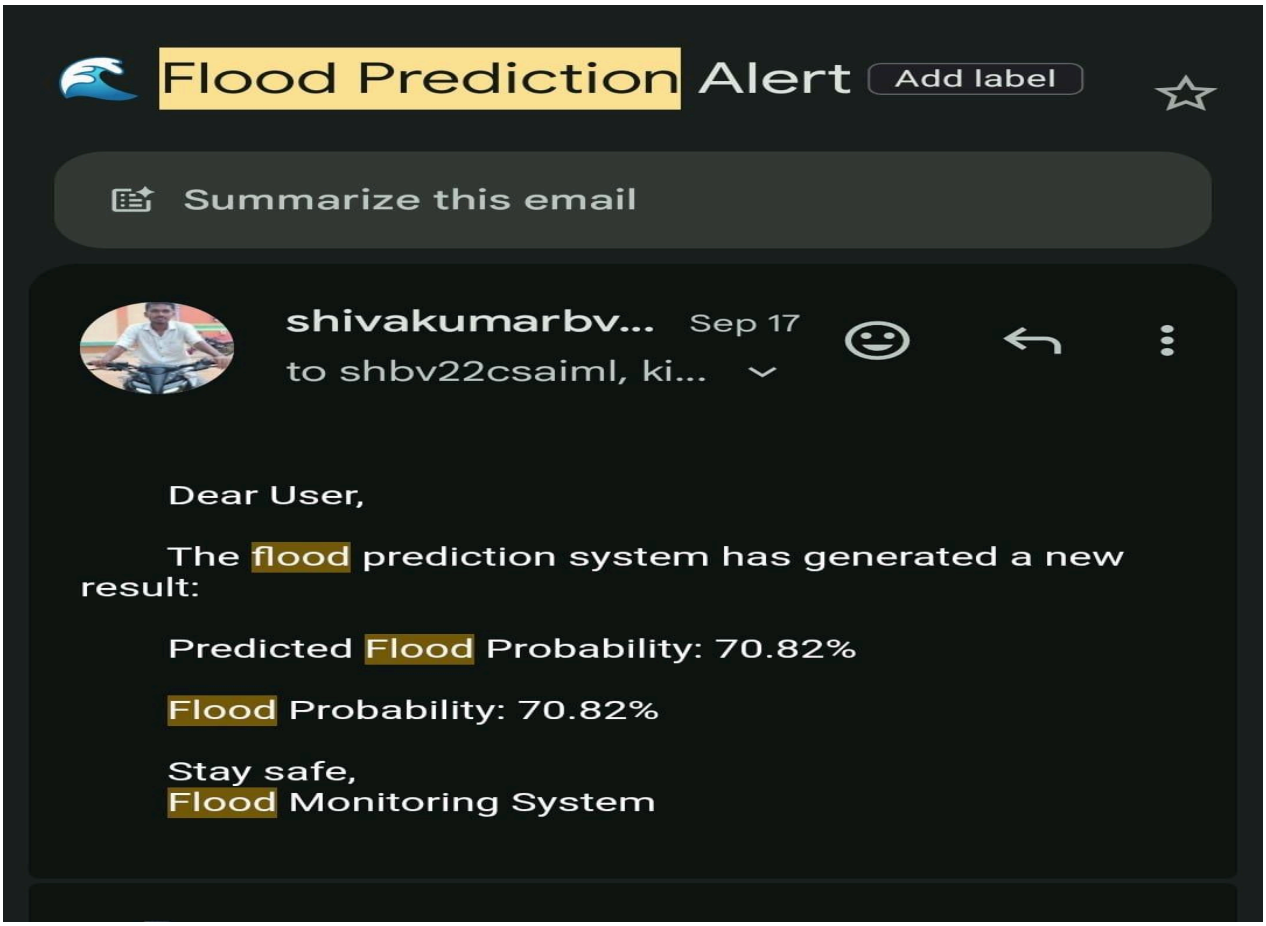

Fig 6.6 Mail Alert

6.6 Discussion

The results of the hybrid CNN–SVM flood detection system indicate that the proposed approach is effective and reliable for classifying flood and non-flood images. The CNN feature extractor successfully learned relevant visual elements such as water patterns, reflections, surrounding structures, and texture variations. Integrating SVM as the final classifier improved decision boundary sharpness and reduced misclassification instances compared to using a fully CNN-based model alone. The system reached an accuracy of approximately **90%**, supported by strong precision, recall, and F1-score values, demonstrating balanced performance across both classes. The ROC curve also showed a high AUC value, indicating excellent separability between the two categories. The confusion matrix confirmed minimal false negatives, which is crucial for applications like flood detection where missed alerts can have serious consequences.

Deployment of the model through an interactive dashboard further validated its practical usability. The prediction response time was efficient, and users could easily upload images and view flood probability results. The alert notification feature added a layer of functional value by informing designated recipients when flood risk exceeded a predefined threshold. Overall, the project successfully demonstrates how combining deep learning and machine learning techniques can enhance disaster prediction and monitoring systems. The hybrid architecture improves accuracy, provides interpretability, and proves suitable for real-time implementation.

6.7. Limitations

Although the system performed well, certain limitations were identified during development and evaluation. The model's performance is dependent on the diversity and quality of the dataset. Images with unusual lighting conditions, low resolution, or partial water visibility occasionally resulted in inaccurate predictions. The dataset size was also moderately limited, meaning the model may struggle when exposed to unfamiliar regional landscapes or seasonal variations.

Additionally, the system is trained to classify only two categories—flood and non-flood—and

does not analyze severity levels or water depth, which could provide more comprehensive insights.

Another limitation is that the current implementation works on static uploaded images and does not process real-time video feed or IoT sensor data. The model also requires further optimization to reduce false positives, which may trigger unnecessary alerts in real-world use. Deployment is currently local and not cloud-based, limiting scalability for large-scale or multi-region deployment. Finally, the model does not self-adapt or retrain automatically; continuous updates would be required as new data becomes available.

CHAPTER 7

CONCLUSIONS AND FUTURE SCOPE

The hybrid CNN–SVM flood detection system developed in this project successfully demonstrates an effective approach for automated image-based flood classification. By combining MobileNetV2 for deep feature extraction with an SVM classifier for final decision-making, the system achieved an accuracy of approximately 90%, confirming the strength of the hybrid architecture over standalone machine learning or deep learning models. The evaluation results, including confusion matrix analysis, ROC curve, precision, recall, and F1-score, indicate balanced and reliable performance across both flood and non-flood image categories.The deployment of the model through an interactive dashboard enabled seamless real-time prediction, making the system user-friendly and practical for real-world use. The integrated alert mechanism further enhanced the system's utility by providing timely notifications during potential flood events, supporting early response and risk mitigation.

Overall, the project demonstrates that machine learning and deep learning can play a significant role in improving disaster monitoring and emergency decision-making. While there are areas for improvement—such as expanding the dataset, enhancing generalization, and integrating live input sources—the system serves as a strong foundation for future advancements in automated environmental monitoring and intelligent flood detection technologies

7.1 Major Findings

- The hybrid CNN–SVM approach achieved approximately 90% accuracy, outperforming a standalone CNN model in classification reliability.
- The model demonstrated strong precision and recall, indicating balanced detection of both flood and non-flood images.
- The confusion matrix showed very few false negatives, meaning the system rarely missed detecting actual flood images, which is critical for safety applications.

- The ROC curve achieved a high AUC score (0.98), confirming excellent separability between classes.
- Model deployment through a dashboard allowed real-time prediction and smooth user interaction.
- The automated email alert system functioned successfully, ensuring timely notification during high flood probability cases.
- Training and validation performance graphs indicated slight overfitting, but the SVM layer helped improve generalization.

7.2. Implications

- The system can be used as a supportive early-warning tool, enhancing disaster preparedness and response strategies.
- The hybrid approach demonstrates that deep learning combined with traditional ML techniques can improve accuracy in complex classification tasks.
- With further optimization and dataset expansion, the model can be scaled for government emergency response systems, smart city monitoring, and environmental agencies.
- The successful alert mechanism implies potential integration with IoT devices, surveillance systems, and real-time monitoring networks.
- The project establishes a foundation for future research on severity detection, live video analysis, and multi-hazard prediction systems.
- The results highlight the importance of data diversity, model tuning, an**d hybrid architectures** in improving AI-based disaster management solutions.

7.3. Future Scope

- **Integration with Real-Time Data Sources:**

  The system can be expanded to support live video feeds from CCTV cameras, drone footage, and IoT-based water-level sensors to provide continuous monitoring instead of relying solely on static images.

- **Cloud-Based Deployment:**

  Deploying the model on cloud platforms such as AWS, Azure, or Google Cloud will enable large-scale usage, remote accessibility, and improved processing speed for multiple simultaneous users.

- **Mobile Application Development:**

  A mobile app version of the system can be developed to allow field workers, emergency teams, and the public to capture and upload images instantly for real-time flood assessment.

- **Flood Severity Classification:**

  The model can be enhanced to not only detect flood presence but also estimate severity levels such as mild, moderate, or severe flooding, supporting more informed decision-making.

- **Dataset Expansion and Retraining:**

  The accuracy and generalization capability of the model can be improved by collecting more diverse datasets covering different terrains, climates, and environmental conditions.

- **Integration with GIS and Weather Systems:**

  Combining the model with geospatial data and weather forecasting tools can help predict flood risk areas, improving disaster preparedness and early warning systems.

- **Self-Learning and Auto-Update Capability:**

  Implementing an automated retraining pipeline will allow the system to learn from new data over time, improving long-term accuracy and adaptability.

REFERENCES

[1] I. Goodfellow, Y. Bengio and A. Courville, Deep Learning. Cambridge, MA: MIT Press, 2016.

[2] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov and L. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), pp. 4510–4520, 2018.

[3] C. Cortes and V. Vapnik, "Support-vector networks," Machine Learning, vol. 20, no. 3, pp. 273–297, 1995.

[4] A. Krizhevsky, I. Sutskever and G. Hinton, "ImageNet classification with deep convolutional neural networks," Advances in Neural Information Processing Systems (NIPS), pp. 1097–1105, 2012.

[5] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," Proc. Int. Conf. Learning Representations (ICLR), pp. 1–14, 2015.

[6] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.

[7] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," Journal of Machine Learning Research, vol. 15, pp. 1929–1958, 2014.

[8] P. William, M. Rajan and S. Kumar, "IoT-based real-time flood prediction using deep learning," IEEE Access, vol. 9, pp. 110562–110573, 2021.

[9] R. Gupta and A. Ranjan, "Machine learning-based flood detection using image processing techniques," International Journal of Advanced Computer Science and Applications, vol. 12, no. 5, pp. 45–52, 2021.

[10] H. Abdellatif et al., "A hybrid deep learning model for disaster image classification," IEEE Transactions on Computational Social Systems, vol. 8, no. 4, pp. 1025–1035, 2021.

[11] S. Patel and P. Mehta, "Improved flood identification using convolutional neural networks,"

International Journal of Engineering and Advanced Technology, vol. 9, no. 6, pp. 42–48, 2020.

[12] UNDRR, "Global Flood Report," United Nations Office for Disaster Risk Reduction, Geneva, 2023.

[13] F. Chollet, "Keras," 2015. [Online]. Available: https://keras.io/.

[14] Scikit-learn Developers, "Scikit-Learn: Machine Learning in Python," 2024. [Online]. Available: https://scikit-learn.org/.

[15] A. Howard et al., "MobileNets: Efficient convolutional neural networks for mobile vision applications," arXiv preprint arXiv:1704.04861, 2017.

APPENDIX

The appendix section contains supplementary materials, supporting content, and reference outputs that complement the main body of the project. While these components are not essential for understanding the primary methodology or results, they provide additional clarity, validation, and transparency regarding the development process. This section includes full-resolution versions of graphs, model outputs, system interface screenshots, and sample dataset images referenced throughout the report. The appendix serves as a detailed record of supporting evidence, helping readers verify the results and understand system behavior more comprehensively.

Appendix A – Dataset Description

- The dataset consists of images collected from open-source repositories, publicly available image datasets, and verified online resources.
    - The images are grouped into two main categories:
      Flood images (showing water overflow, flooded streets, buildings, and natural disaster scenarios)
    - Non-Flood images (showing normal environmental conditions with no excess water accumulation)
- A total of **10,000 images** were used in the project.
- The dataset is balanced with:
    - 5,039 Flood images
    - 4,961 Non-Flood images
- All images were preprocessed by resizing to 128 × 128 pixel**s** for model compatibility.
- Pixel values were normalized to improve model learning efficiency and reduce noise.
- The dataset was split into:
    - 80% Training data
    - 20% Testing data
- The images represent diverse environments including streets, residential areas, bridges, farmland, and natural landscapes.

- The dataset includes variations in weather, lighting conditions, camera angles, and geographic locations to improve generalization.

- The dataset serves as the core input for training the hybrid CNN–SVM flood detection model.

Appendix B – System Output Screenshots

B.1 Flood Prediction Output Example

The screenshot below shows an example of the system detecting a flood condition after an image is uploaded to the dashboard. The selected image is processed through the hybrid CNN–SVM model, and the prediction result is displayed along with the calculated probability score. In this example, the system identifies the image as "Flood Detected" with a probability of 13.35%. The alert mechanism highlights the result with a warning message, demonstrating how the system communicates risk levels to the user.

Fig B.1 Dashboard Output

B.2 Automated Email Alert Notification

The image shown below represents the automated alert notification generated by the system when a flood condition is detected. This notification is sent through an SMTP-based mailing service configured

during the deployment phase. The system continuously evaluates uploaded images and, when the prediction probability surpasses the predefined safety threshold, the alert mechanism is triggered. The email contains essential information, including the predicted probability value, system status, and a

cautionary message instructing the user to take necessary precautions.

This feature ensures that users receive timely warnings, even if they are not actively monitoring the dashboard. It adds a layer of practicality and real-world applicability, making the system suitable for deployment in communities, government disaster response units, or monitoring agencies. By delivering alerts directly to the user's inbox, the system supports faster decision-making and can significantly aid preparedness during flood-risk situations.

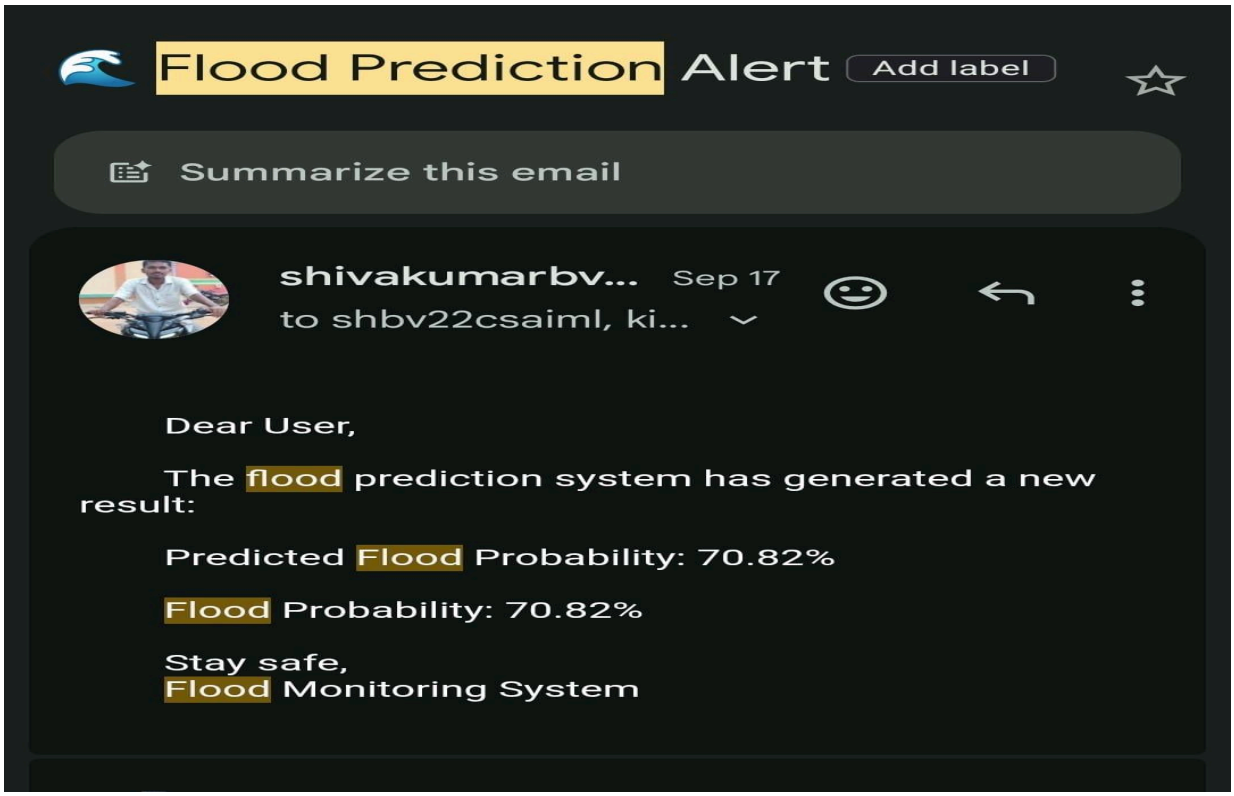

Fig B.2 Mail Alert System Output